

BBRv2+: Towards balancing aggressiveness and fairness with delay-based bandwidth probing

Furong Yang^{a,b,c}, Qinghua Wu^{a,b,d}, Zhenyu Li^{a,b,d,*}, Yanmei Liu^e, Giovanni Pau^{f,g}, Gaogang Xie^h

^a Institute of Computing Technology, Chinese Academy of Sciences, China

^b University of Chinese Academy of Sciences, China

^c Sorbonne University, France

^d Purple Mountain Laboratories, China

^e Alibaba Group, China

^f University of Bologna, Italy

^g University of California, Los Angeles, USA

^h Computer Network Information Center, Chinese Academy of Sciences, China

ARTICLE INFO

Keywords:

Congestion control

BBR

BBRv2

ABSTRACT

BBRv2, proposed by Google, aims at addressing BBR's shortcomings of unfairness against loss-based congestion control algorithms (CCAs) and excessive retransmissions in shallow-buffered networks. In this paper, we first comprehensively study BBRv2's performance under various network conditions and show that BBRv2 mitigates the shortcomings of BBR. Nevertheless, BBRv2's benefits come with several costs, including slow responsiveness to bandwidth dynamics and low resilience to random losses. We then propose BBRv2+ to address BBRv2's performance issues without sacrificing its advantages over BBR. To this end, BBRv2+ uses delay information to cautiously guide the aggressiveness of its bandwidth probing. In doing so, it achieves fast responsiveness to bandwidth dynamics and fairness against loss-based CCAs at the same time. BBRv2+ also integrates mechanisms for improved resilience to random losses and network jitter. Extensive experiments demonstrate the effectiveness of BBRv2+. Specifically, it achieves 25% higher throughput and comparable queuing delay in comparison with BBRv2 in high-mobility network scenarios.

1. Introduction

Congestion control has been one of the active research topics in computer networks since it was introduced in the 1980s [5]. More than three decades of research on congestion control have brought us a plethora of congestion control algorithms (CCAs) and TCP variants, aiming at efficient utilization of available bandwidth while fairly sharing the bottleneck bandwidth among multiple flows. For instance, Linux kernel alone has more than 15 different CCAs [6]. While we see many recent proposals on learning-based CCAs (e.g. Remy [7], Aurora [8], PCC-Vivace [9], Indigo [10], Orca [6]), the widely deployed CCAs today are still classic ones (e.g. Cubic [11], BBR [12]).

In this paper, we focus on BBR and its upgrade, BBRv2, as BBR has been used by 22% of the Alexa Top 20K websites [13] and BBRv2 will likely replace BBR in the near future.¹ BBR is a rate-based CCA that sets its sending rate based on the measured bottleneck bandwidth (*BtlBW*) and round trip propagation time (*RTprop*). That said, instead of

reacting to congestion signals such as packet losses or delay dynamics, BBR tries to actively operate at Kleinrock's optimal operating point [14] to maximize throughput without incurring standing queues at the bottleneck link. Previous empirical studies [3,4,15–18] have disclosed several shortcomings of BBR: (1) it introduces excessive retransmissions in shallow-buffered networks²; (2) it is not fair when competing with flows using loss-based CCAs (e.g. Cubic) and BBR flows with different Round Trip Times (RTTs); (3) its performance degrades when network jitter is high.

To address these issues in BBR, Google proposed BBRv2 [19] that inherits most of the design principles from BBR, but reacts to packet losses and Explicit Congestion Notification (ECN) marks for better co-existence with loss-based CCAs and being less aggressive in shallow-buffered networks. Given that BBRv2 may eventually replace BBR, understanding how BBRv2 actually performs is of great importance for its deployment. We have also seen several studies [2,20–23] on

* Corresponding author.

E-mail address: zyli@ict.ac.cn (Z. Li).

¹ As of March 2021, Google had finished the roll-out of BBRv2 for internal TCP traffic [1].

² According to previous studies [2–4] a buffer smaller than $2 \times \text{BDP}$ (Bandwidth Delay Product) is considered shallow.

measuring BBRv2, which have shown that, in comparison with BBR, BBRv2 improves inter-protocol fairness against loss-based CCAs and reduces retransmissions in shallow-buffered networks. Nevertheless, we found in this paper, these improvements come with several costs, including low resilience to random packet losses and slow responsiveness to bandwidth dynamics.

In this paper, we first evaluate BBRv2 in various network conditions with an emphasis on the reasons behind the observed performance issues. Our key observations from the empirical study of BBRv2 are as follows:

- Due to its conservative strategies in bandwidth probing and congestion window (*cwnd*) estimation, BBRv2 achieves better inter-protocol fairness against loss-based CCAs in shallow-buffered networks than BBR. On the other hand, these strategies also make BBRv2 less competitive than BBR in terms of throughput under moderate buffers. BBRv2 also improves fairness among flows of different RTTs, compared with BBR.
- In shallow-buffered networks, BBRv2 significantly reduces retransmissions, compared with BBR. However, the throughput of BBRv2 is 13%~16% lower than that of BBR. This is because BBRv2 limits its *cwnd* to about $0.85 \times$ Bandwidth Delay Product (BDP) for most of the time in these networks.
- BBRv2 is less resilient to random packet losses than BBR. Interestingly, we find that carefully tuning the loss threshold parameter in BBRv2 according to bottleneck buffer sizes can enhance BBRv2's loss resilience without sacrificing its advantages in retransmission and fairness.
- BBRv2 is less responsive to bandwidth dynamics than BBR, which leads to low bandwidth utilization and high queuing delay in networks with bandwidth dynamics. The long bandwidth probing interval and the long expiry time of *BtlBW* estimation are the two major contributors.
- Like BBR, BBRv2's performance still suffers from *cwnd* exhaustion in high-jitter networks that are not rare in wireless scenarios [17, 24–27].

The results of our empirical study of BBRv2 raise an important question: *are we able to improve BBRv2's performance while keeping its advantages in retransmission and fairness? If so, how do we achieve this goal?*

BBRv2's shortcomings lie in lower loss resilience and slower responsiveness to bandwidth dynamics. The issue of loss resilience can be mitigated by carefully tuning the loss threshold parameter in BBRv2. But, the aggressiveness of BBRv2 in bandwidth probing needs to be adjusted to improve its responsiveness to bandwidth dynamics. Currently, the aggressiveness of bandwidth probing in BBR and BBRv2 is either hard-coded or pre-configured according to designers' experience without the perception of the network environment. Such unguided aggressiveness may make the bandwidth probing either over-aggressive (like BBR) or over-conservative (like BBRv2) in certain environments. Thus, we argue that BBRv2 needs to consider the feedback from the network environment to dynamically adjust the aggressiveness of bandwidth probing.

To address the above gap, we propose BBRv2+. Firstly, BBRv2+ integrates delay information into its path model, which serves as the feedback to guide its aggressiveness in bandwidth probing. Secondly, BBRv2+ partially redesigns the state-machine of BBRv2 in order to utilize the delay information to guide the aggressiveness of BBRv2+'s bandwidth probing. In doing so, BBRv2+ balances between the aggressiveness in bandwidth probing and the fairness against loss-based CCAs. Thirdly, to avoid being suppressed when co-existing with loss-based CCAs in deep-buffered networks because of using the delay information, BBRv2+ incorporates a dual-mode mechanism—it dynamically switches between BBRv2+'s state-machine and BBRv2's state-machine

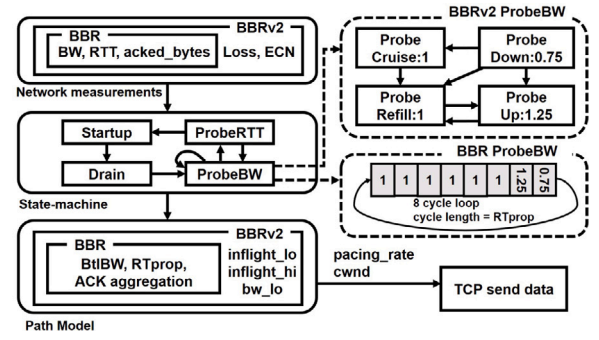


Fig. 1. Architecture of BBR and BBRv2.

depending on whether loss-based CCAs co-exist. Finally, BBRv2+ addresses the *cwnd* exhaustion problem in high-jitter networks by compensating its estimated BDP according to observed jitter; the compensation mechanism improves the estimation accuracy of BDP in high-jitter networks³.

We conduct extensive experiments in networks emulated by Mininet [28] and Mahimahi [29] with real-world traces. The results show that compared with BBRv2, BBRv2+ succeeds to balance the aggressiveness of bandwidth probing and the fairness against loss-based CCAs, improves the resilience to network jitter, and achieves 25% higher throughput and comparable queuing delay in high-mobility scenarios where the bandwidth is very dynamic.

To sum up, the contributions of this paper are three-fold: (1) we make a detailed investigation of BBRv2 to examine its benefits and shortcomings in comparison with BBR; (2) we propose BBRv2+ to address the shortcomings of BBRv2 while barely sacrificing BBRv2's advantages; (3) we show through extensive experiments that BBRv2+ meets its design goal. We open-source BBRv2+ to the research community for further tests and improvements [30].

The remainder of this paper is organized as follows. We first give an overview of BBR and BBRv2 in Section 2. Then, a deep dive into BBRv2 is presented in Section 3. Next, Section 4 describes the design and implementation of BBRv2+, followed by the evaluation of BBRv2+ in Section 5. We present the related work in Section 6. Finally, the paper is concluded in Section 7.

2. Background

2.1. An overview of BBR

BBR is designed to operate near Kleinrock's optimal point [14]. Operating at this optimal point depends on accurate measurements of *BtlBW* and *RTprop* (as $BDP = BtlBW \times RTprop$). Since these two variables cannot be measured simultaneously, BBR introduces a state-machine-based method that alternatively estimates *BtlBW* and *RTprop*.

The overall architecture of BBR is illustrated in Fig. 1. BBR takes a set of measurements as the inputs of its state-machine, builds a path model based on the state-machine, and outputs control parameters to a TCP sending engine that controls how much data can be sent (i.e. setting *cwnd*) and at what speed (i.e. setting *pacing_rate*). The path model of BBR includes *BtlBW* (stored in a 10-RTT-windowed *max_filter* that keeps bandwidth measurements in recent 10 RTTs and returns the maximum), *RTprop* (calculated as the minimum RTT measured in recent 10 s), and ACK aggregation degree (for compensating *cwnd* when network paths have severe ACK decimation, e.g. Wi-Fi paths). The output parameters, *cwnd* and *pacing_rate*, are computed based on $cwnd = cwnd_gain \times BDP$ and $pacing_rate = pacing_gain \times BtlBW$

³ This mechanism can also be applied to BBR and BBRv2.

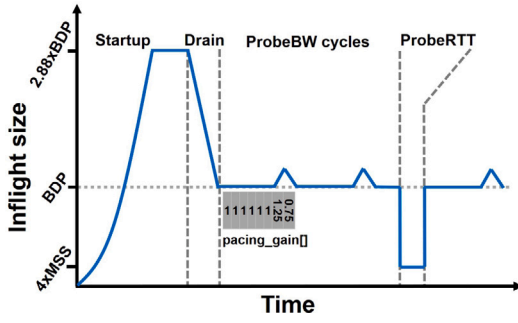


Fig. 2. Illustration of BBR life-cycle.

respectively. BBR sets its *cwnd_gain* and *pacing_gain* according to which state it operates in to control whether to probe for more available bandwidth, or drain the buffer at a bottleneck link, or cruise at the speed of *BtlBW*.

As illustrated in Fig. 2, there are four states in the BBR life-cycle. Firstly, BBR starts with the Startup state, in which it exponentially increases the *cwnd* size and sending rate by setting *pacing_gain* and *cwnd_gain* to $2/\ln(2)$. BBR transits to the Drain state if it is in the plateau of *BtlBW* for three RTTs. In the Drain state, BBR reduces *pacing_gain* to $\ln(2)/2$ to drain the standing queue at the bottleneck link induced during Startup. After the above two stages, BBR has successfully built the path model, with *RTprop* measured at the beginning of Startup and *BtlBW* measured at the end of Startup. After that, BBR switches to a steady phase where BBR alternatively runs in the ProbeBW and ProbeRTT state. During the ProbeBW state, BBR sets *pacing_gain* to 1.0 to cruise at the Kleinrock's optimal point for 6 cycles, then sets *pacing_gain* to 1.25 to explore more bandwidth for 1 cycle, and thereafter sets *pacing_gain* to 0.75 to drain the possible standing queue for 1 cycle. It is noteworthy that BBR always sets *cwnd_gain* to 2.0 during the ProbeBW state to allow sending an extra BDP-sized data during loss recovery. If BBR discovers that its *RTprop* has not been updated for 10 s, it enters the ProbeRTT state. During the ProbeRTT state, BBR reduces its inflight size to $4 \times \text{MSS}$ (Max Segment Size) and waits for $\max\{RTT, 200 \text{ ms}\}$ to obtain an updated value of *RTprop*.

2.2. An overview of BBRv2

As observed in numerous previous studies [3,4,15–17], BBR has two key issues: unfairly sharing bandwidth with loss-based CCAs and high retransmission rates in shallow-buffered networks. The reasons behind these issues are that BBR is congestion signal agnostic and is over-aggressive when probing for more bandwidth. To mitigate the problems above, Google proposed BBRv2 [19], which inherits most of BBR's design yet redesigns the ProbeBW state, as shown in Fig. 1.

To avoid being congestion signal agnostic like BBR, BBRv2 reacts to packet loss and ECN marks [31] for building a tight estimation of the capacity of a bottleneck link. Specifically, BBRv2 adds three variables (*inflight_lo*, *inflight_hi*, and *bw_lo*). *inflight_lo* and *bw_lo* are the short-term lower bounds of inflight size (i.e. *cwnd*) and sending rate respectively, in order to capture the temporary status of network paths (e.g. cross-traffic takes a share of capacity); *inflight_hi*, on the other hand, is the long-term upper bound of *cwnd* to reduce the likelihood of packet losses.

BBRv2 calculates *pacing_rate* and *cwnd* as in Eq. (1). The reason for having non-zero headroom values is to reserve spare capacity for other flows to quickly grab some bandwidth, improving its friendliness [32]. As bounded by *inflight_lo* and *inflight_hi*, the estimation of *cwnd* in BBRv2

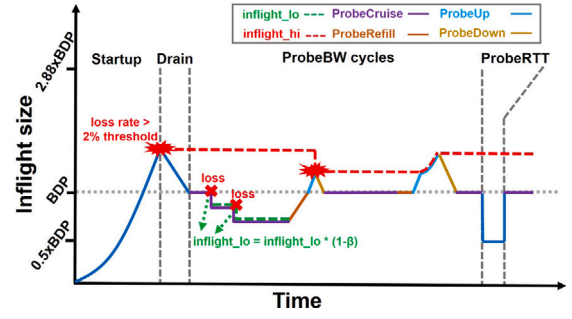


Fig. 3. Illustration of BBRv2 life-cycle.

is more conservative than that of BBR.

$$pacing_rate = pacing_gain \times \min(BtlBW, bw_lo)$$

$$cwnd = \min((1 - headroom) \times inflight_hi,$$

$$inflight_lo, cwnd_gain \times BDP)$$

(1)

$$headroom = \begin{cases} 0.15, & \text{ProbeCruise or ProbeRTT} \\ 0, & \text{Otherwise} \end{cases}$$

The BBRv2 life-cycle is shown in Fig. 3. Firstly, besides the condition that BBR uses, BBRv2 also exits from Startup if the current loss/ECN mark rate is above a predefined threshold (e.g. 2% for the current loss rate). If BBRv2 exits from Startup because of a high loss/ECN mark rate, it sets *inflight_hi* to the current inflight size, otherwise, *inflight_hi* remains at its initial value ($+\infty$). Secondly, BBRv2 decomposes BBR's ProbeBW state into four sub-states, ProbeCruise, ProbeRefill, ProbeUp, and ProbeDown.

ProbeCruise: In the ProbeCruise state, BBRv2 sets *pacing_gain* to 1.0. If any loss or ECN mark occurs during this state, BBRv2 updates *inflight_lo* and *bw_lo* to $\max\{(1-\beta) \times inflight_lo, BtlBW_{curr}\}$ and $\max\{(1-\beta) \times bw_lo, inflight_{curr}\}$ respectively, where *BtlBW_{curr}* and *inflight_{curr}* are the current measurements of bandwidth and inflight size. It is noteworthy that β is either pre-configured⁴ or calculated based on ECN mark rates.

ProbeRefill: When BBRv2 has been in ProbeCruise for a period of *T* (*T* is determined in ProbeDown), BBRv2 transits to ProbeRefill, by setting *inflight_lo* and *bw_lo* to $+\infty$ to refill the “pipe” with BDP-sized inflight data, which lasts for one RTT. The goal of this state is to avoid early packet losses before the capacity is fully utilized in shallow-buffered networks since BBRv2 will accelerate in the following ProbeUp state.

ProbeUp: In ProbeUp, BBRv2 sets *pacing_gain* to 1.25 to probe for more available bandwidth. If BBRv2 has increased its inflight size all the way up to *inflight_hi* without seeing a loss/ECN mark rate exceeding the threshold, it starts to exponentially increase *inflight_hi*. This state ends either when the current loss/ECN mark rate exceeds the threshold, or when the inflight size reaches $1.25 \times BDP$ for at least one *RTprop*. In the former case, BBRv2 sets *inflight_hi* to the current inflight size.

ProbeDown: In ProbeDown, BBRv2 drains the potential queue at the bottleneck link by setting *pacing_gain* to 0.75. BBRv2 also sets the duration (*T*) for the next ProbeCruise state to $\min\{rand(2, 3), \frac{BDP}{MSS} \times RTT\}$ s, where *rand*(2, 3) means a number between two and three. The intention of *T* is to match the interval between loss recovery epochs of Reno for TCP fairness. It is noteworthy that *T* is usually longer than the bandwidth probing interval of BBR ($8 \times RTprop$). Therefore, BBRv2 probes for bandwidth in a more-conservative way. The ProbeDown state ends when BBRv2 cuts its inflight size below $\min\{BDP, 0.85 \times inflight_hi\}$. Thereafter, BBRv2 transits to the next ProbeCruise state.

⁴ The default value 0.3 is for BBRv2 to co-exist with Cubic [33].

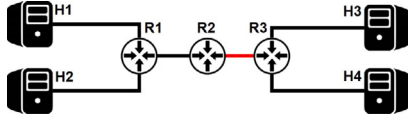


Fig. 4. Mininet testbed: the bottleneck link is the link between R2 and R3 (marked in red); delay and packet loss are configured on the link between R1 and R2; network jitter is configured at R3's outgoing interfaces towards H3 and H4.

Moreover, BBRv2 no longer uses a 10-RTT-windowed *max_filter* for the estimation of *BtlBW*; rather, it takes the maximum bandwidth measured in the recent two ProbeBW stages as the estimated *BtlBW*.

Summary of BBRv2: BBRv2 is more conservative in *cwnd* estimation and bandwidth probing than BBR, in order to reduce the excessive retransmissions in shallow-buffered networks and improve the fairness of BBR. However, incorporating loss or ECN information in BBRv2 degrades its performance under random losses. More importantly, because of less frequent bandwidth probing in BBRv2, it may not respond to bandwidth dynamics as quickly as BBR. We will quantify the benefits and shortcomings of BBRv2 in the next section.

3. A deep dive into BBRv2

In this section, we empirically investigate the improvements and overheads of BBRv2, in comparison with BBR. Our key observations include: (1) BBRv2 improves inter-protocol fairness and RTT fairness, and also reduces retransmissions in shallow-buffered networks; this observation reaffirms the findings in the previous studies [2,20–23]; (2) the improvements of BBRv2 come with the cost of low resilience to random loss and slow responsiveness to bandwidth dynamics. That said, it fails to achieve a balance between the aggressiveness in bandwidth probing and the fairness against loss-based CCAs; (3) like BBR, BBRv2 experiences low throughput in high-jitter networks because of underestimation of BDP.

3.1. Methodology

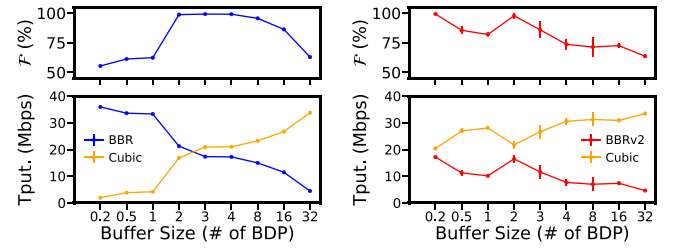
We rely on Mininet [28] that runs on a server with 8 Intel Xeon Platinum cores and 32 GB memory for empirical analysis. The operating system is Ubuntu 18.04.5 with BBR and BBRv2 [33] installed. Fig. 4 shows the network topology. Linux *tc-netem* [34] is used to emulate different network conditions by varying buffer size, link speed, delay, random loss rates, jitter. We use *iPerf3* [35] to generate TCP traffic between senders and receivers. All packets are captured via *tcpdump* [36] and analyzed by *tcptrace* [37] to get various performance metrics (e.g. RTTs, throughput, retransmissions, inflight bytes). Besides, the values of internal variables (e.g. *cwnd*, *pacing_rate*, *RTTprop*, *BtlBW*) in BBR and BBRv2 are recorded in Linux kernel logs. Finally, the queue length at the bottleneck link between R2 and R3 is reported by *tc* [38]. As in previous studies [3,4,15] did, each set of experiments is repeated five times and the average results (along with variations if possible) are reported.

3.2. Fairness

We investigate two types of fairness: the inter-protocol fairness against loss-based CCAs and RTT fairness. In this set of experiments, two flows start simultaneously, one from H1 to H3 and the other from H2 to H4; both flows last for three minutes for the throughput convergence. The bottleneck bandwidth was fixed at 40 Mbps without network jitter or random packet losses.

We use Jain's fairness index [39] (F) of the two flows to evaluate the fairness. The index is calculated as:

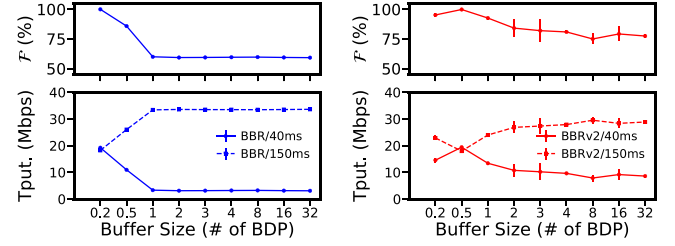
$$F = \frac{(T_1 + T_2)^2}{2 * (T_1^2 + T_2^2)} \quad (2)$$



(a) BBR vs Cubic

(b) BBRv2 vs Cubic

Fig. 5. Inter-protocol fairness of BBR/BBRv2 under different buffer sizes (markers: averages, bars: standard deviations).



(a) BBR

(b) BBRv2

Fig. 6. RTT fairness of BBR/BBRv2 under different buffer sizes (markers: averages, bars: standard deviations).

where T_i is the i th flow's average throughput over the last 60 s of the flow's lifetime. $F = 1$ indicates the maximum fairness where two flows have identical average throughput, and $F = 0.5$ represents that one flow's throughput is zero and the fairness is minimized. We also vary the bottleneck buffer size as it also impacts fairness results.

3.2.1. Inter-protocol fairness

In the experiment, the flow from H1 to H3 uses either BBR or BBRv2, and that from H2 to H4 uses Cubic, which is the default CCA in Linux. The RTTs of the two paths are 40 ms.

Fig. 5 shows the inter-protocol fairness results for both BBR and BBRv2, where we vary the buffer size (expressed in multiples of BDP) at the bottleneck link. We can observe that compared with BBR, BBRv2 significantly improves Jain's fairness index when the buffer is shallow (i.e., less than $2 \times$ BDP). This is due to the fact that BBRv2 reacts to losses caused by buffer overflow and bounds the inflight size by using *inflight_hi* and *inflight_lo*. When the bottleneck buffer gets deeper, Cubic obtains more bandwidth than BBR or BBRv2. This is because that the inflight size of both BBR and BBRv2 is upper-bounded by about $2 \times$ BDP, while Cubic's inflight size can go beyond this value under deep buffers. As the two flows experience similar RTTs, a larger inflight size means higher throughput. We also note that BBRv2 is less competitive than BBR under moderate buffers. The reason lies in the fact that BBRv2 is more conservative in bandwidth probing and *cwnd* estimation.

3.2.2. RTT fairness

In this experiment, two flows use the same CCA, either BBR or BBRv2. The RTT is 40 ms for the path between H1 and H3, and 150 ms for the path between H2 and H4.

We can see from Fig. 6 that when the buffer is quite shallow (i.e., $0.2 \times$ BDP), BBR has good RTT fairness. However, when the buffer size becomes larger, the BBR flow with longer RTT gradually occupies all the bandwidth and starves the flow with shorter RTT. The reason for the poor RTT fairness of BBR is well documented by the previous studies [4,18]. Specifically, the aggregated sending rate of two BBR flows exceeds bottleneck bandwidth because of bandwidth probing. As such, a persistent queue forms at the bottleneck link. As the *cwnd*

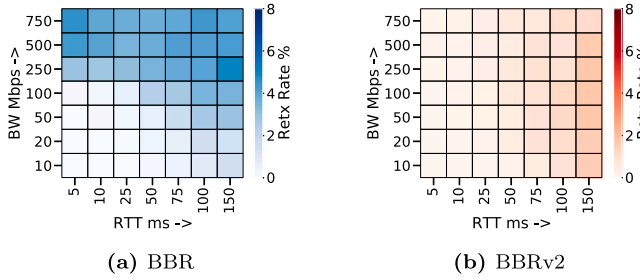


Fig. 7. The heatmap of the retransmission rate of BBR or BBRv2 under various network conditions.

of BBR is proportional to RTT_{prop} , the flow with longer RTT pours more data into the bottleneck buffer, thus having a larger share of the bottleneck link's capacity. This problem is not severe under shallow buffers, as excess packets are mostly dropped instead of forming a persistent queue.

Compared with BBR, BBRv2 has better RTT fairness, especially under deep buffers. The reason is three-fold. First, when the buffer size is moderate, losses are triggered due to buffer overflow, and then both flows reduce their $cwnd$ proportional to BDP. As the flow with longer RTT has a larger BDP, it reduces its $cwnd$ more than the flow with shorter RTT. Second, when BBRv2 flows are cruising at the speed of $BtlBW$, they always try to leave headroom for other flows to explore the bandwidth. Third, BBRv2 flows yield occupied capacity more frequently because it enters the ProbeRTT state more often than BBR.

3.3. Retransmission and throughput

As one of BBRv2's design goals is to reduce unnecessary retransmissions in shallow-buffered networks, next we investigate whether BBRv2 achieves this design goal. The experimental setup is similar to that in the previous work [15], where the bottleneck bandwidth varies in 10~750 Mbps and the path RTT varies in 5~150 ms as these values are commonly employed in modern networks [4,15,40]. The buffer size at the bottleneck link is set to 100 KB to emulate shallow-buffered networks.⁵ For each setup, we launch a TCP flow for 30 s and record its retransmission rate.

The heatmaps in Fig. 7 show the retransmission rates of BBR and BBRv2 under various network conditions. We can observe that the retransmission rate of BBRv2 is significantly reduced compared with that of BBR, especially when the BDP is larger than 400 KB (i.e. the buffer size $\leq 0.25 \times BDP$).

The lower retransmission rate of BBRv2 in shallow-buffered networks stems from the fact that BBRv2 reacts to packet losses, while BBR does not. In the Startup and ProbeUp state, BBRv2 tries to send at a rate higher than the bottleneck bandwidth, which leads to excessive losses (i.e. loss rate $\geq 2\%$). The excessive losses trigger $inflight_{hi}$ to be set as the current inflight size that is likely close to BDP in shallow-buffered networks. As shown in Eq. (1), BBRv2's $cwnd$ is bounded below $0.85 \times inflight_{hi}$ in ProbeCruise. Since BBRv2 flows spend most of their time in ProbeCruise, the average throughput of BBRv2 is expected to be 15% lower than the available bandwidth in shallow-buffered networks.

That said, BBRv2 trades off throughput against retransmission in shallow-buffered networks. To verify this, we compute the throughput gain of BBR over BBRv2 ($Tput_Gain$), which is defined in Eq. (3), where

⁵ 100 KB is less than the BDP of most bandwidth-RTT combinations in our setup.

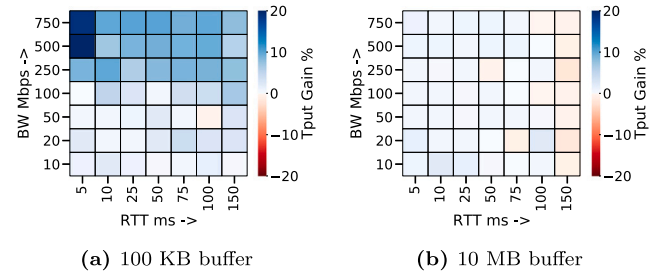


Fig. 8. The heatmap of $Tput_Gain$ (in percentage): (a) shallow-buffered network, (b) deep-buffered networks.

$Tput_{BBR}$ (resp. $Tput_{BBRv2}$) is the average throughput of a BBR (resp. BBRv2) flow over 30 s.

$$Tput_Gain = \frac{Tput_{BBR} - Tput_{BBRv2}}{Tput_{BBRv2}} \quad (3)$$

From Fig. 8(a), we can observe that under the network conditions where BBRv2 reduces the retransmission rate (when BDP exceeds 400 KB), BBRv2 achieves lower throughput than BBR. Specifically, the throughput of BBRv2 is 13%~16% lower than that of BBR in these cases. In deep-buffered networks, however, packet losses are much less often. It is thus expected that the throughput of BBR and BBRv2 are comparable. This is confirmed by the results in Fig. 8(b), where the buffer size is configured at 10 MB, larger than the BDP of most of the bandwidth-RTT combinations in our setup. The throughput differences between BBR and BBRv2 are indeed marginal in these networks.

3.4. Resilience to random losses

Several early tests [2,21,23] have shown that BBRv2 is less resilient to random losses than BBR, since BBRv2 limits its $cwnd$ by $inflight_{lo}$ and $inflight_{hi}$, which both react to all types of losses. In BBRv2, there are two parameters that decide how $inflight_{lo}$ and $inflight_{hi}$ react to losses. One is the explicit loss threshold (α) and the other one is the $inflight_{lo}$ reduction factor (β). To study how α and β impact BBRv2's loss resilience, we investigate BBRv2 variants with different α and β under random losses, where each specific BBRv2 variant is referred as $BBRv2(\alpha, \beta)$. For α , we cap it at 20% to match the maximum loss rate that BBR can tolerate; for β , we only evaluate the difference between the case with (i.e. $\beta = 0.3$ by default) and without it (i.e. $\beta = 0$). In this set of experiments, the bottleneck bandwidth is set to 40 Mbps, and the path RTT is 40 ms. The buffer size is $32 \times BDP$ to avoid packet losses due to buffer overflow. The random loss rate ranges from 0% to 30%.

Fig. 9 reports the average throughput of each CCA against random loss rates. We observe that the throughput of BBRv2 drops significantly when the random loss rate goes beyond 2%. Moreover, as α increases, the loss resilience of BBRv2 is improved. For instance, with a 10% random loss rate, $BBRv2(20\%, 0.3)$ occupies around half of the maximum bandwidth while the default BBRv2's throughput nearly drops to zero. The impact of β is also remarkable: the loss resilience of $BBRv2(20\%, 0.3)$ is lower than that of $BBRv2(20\%, 0)$, which performs similar to BBR.

The above results indicate that BBRv2's loss resilience can be improved via raising α . Yet, there is a concern on its effectiveness in alleviating the retransmission issue in shallow-buffered networks, because BBRv2 reduces its $cwnd$ when the loss rate exceeds α (see Section 3.3).

To investigate the aforementioned concern, we extend the experiments by considering more BBRv2 variants ($\alpha \in [2\%, 100\%]$, $\beta = 0.3$) and more configurations on buffer sizes (buffer size $\in \{0.2, 0.5, 1.0, 1.5, 2.0\} \times BDP$). Fig. 10 plots the retransmission rates of all considered BBRv2 variants under a random loss rate of 0% (to eliminate the impact of random losses on retransmission rates). Two observations

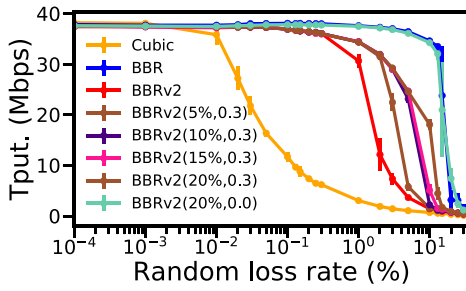


Fig. 9. Average throughput against different random loss rates (buffer size = $32 \times \text{BDP}$).

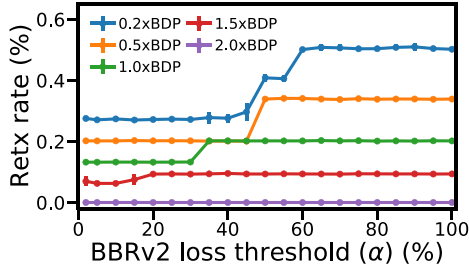


Fig. 10. Retransmission rates versus α values under networks with different buffer sizes. The errorbars in the figure represent the standard deviations of retransmission rate. β is fixed at 0.3 for all experiments; the default value of α in BBRv2 is 2% (the first data point of every line).

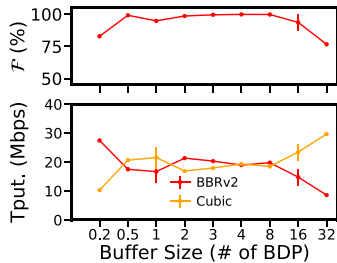


Fig. 11. Inter-protocol fairness of BBRv2(20%, 0.3) (markers: averages, bars: standard deviations).

are notable. Firstly, we observe that the retransmission rate increases when α exceeds a certain point, which depends on the bottleneck buffer size. α values beyond the turning points are too high to be reached by temporary loss rates, thus, limiting the effect of *inflight_{hi}*. Secondly, if the buffer size is large enough (i.e. $2 \times \text{BDP}$ in our experiments), the retransmissions are eliminated, so the value of α becomes irrelevant.

Another concern about lifting α is the impact on the inter-protocol fairness because the larger α is, the slower reaction of BBRv2 to losses is, which makes BBRv2 more aggressive to loss-based CCAs. To investigate this concern, we test the inter-protocol fairness of BBRv2(20%, 0.3) using the same setup as in Section 3.2.1, and plot the results in Fig. 11. In comparison with Fig. 5(b), the inter-protocol fairness of BBRv2 is indeed worsened in the case of extremely shallow buffer ($0.2 \times \text{BDP}$) due to the increased aggressiveness caused by a larger α . Nevertheless, the fairness index is improved under moderate buffers because the increased aggressiveness also makes BBRv2 less vulnerable to Cubic when the bottleneck buffer becomes larger.

Summary of random loss resilience: The loss resilience of BBRv2 can be improved by raising the loss threshold, α . Nevertheless, α should be carefully tuned according to the bottleneck buffer size in order to avoid increasing retransmissions and being too aggressive to loss-based CCAs in extremely shallow-buffered networks.

3.5. Responsiveness to bandwidth dynamics

Next, we investigate BBRv2's responsiveness to bandwidth changes. The experiments are designed as follows. The bandwidth of the bottleneck link is configured to increase or decrease 5 Mbps every 2 s, the path delay is set to 40 ms, and the buffer size is set to $32 \times \text{BDP}$. The internal variables during flow transmission (including *pacing rate* and *BtlBW*, instantaneous throughput, and the queue length at the bottleneck link) are sampled at an interval of 100 ms.

Fig. 12 shows how BBR and BBRv2 adapt to bandwidth increases or decreases respectively. In this figure, the upward and downward spikes of queue length correspond to the actions of BBR/BBRv2 in probing for more bandwidth or draining the bottleneck buffer. We can observe that BBRv2 is less effective than BBR in terms of responsiveness to bandwidth dynamics, resulting in the low utilization of bandwidth and long queuing delay.

As we discussed in Section 2.2, to match the interval between two Reno loss recovery epochs for better inter-protocol fairness, BBRv2 sets its probing interval to $\min\{\text{rand}(2, 3), \frac{\text{BDP}}{\text{MSS}} \times \text{RTT}\}$ s. This interval can be tens of RTTs, which is too conservative in dynamic environments. That said, BBRv2 improves inter-protocol fairness, at the cost of poorer responsiveness to bandwidth dynamics.

3.6. Resilience to network jitter

Several works [17,27] have shown that throughput collapse occurs when BBR operates in high-jitter networks that are widely deployed, e.g. WiFi and 5G networks operating in mmWave band [17,24,25], and cellular networks [17,26], especially when high-mobility involves such as high-speed rails [27]. It is interesting to investigate whether BBRv2 operates well in networks with high jitter.

In this experiment, the bottleneck bandwidth is 40 Mbps and the path RTT is 40 ms. The bottleneck buffer size is set to $32 \times \text{BDP}$ to avoid buffer overflow. To emulate jitter, *tc* is used to add jitter following Gaussian distribution at R3's interface that connects to H3. The mean value of the Gaussian distribution varies from 0~120 ms to emulate different degrees of jitter.

Fig. 13 shows the average value and the standard deviation of throughput of Cubic, BBR, and BBRv2 under various levels of jitter. Compared with Cubic, both BBR and BBRv2 experience low throughput under high jitter. As documented by Kumar et al. [17], BBR underestimates *RTprop* in such networks because it uses the minimum RTT of last 10 s to approximate *RTprop*, which causes *cwnd* exhaustion. This problem still exists in BBRv2, even if BBRv2 updates *RTprop* $2 \times$ frequently than BBR (i.e. BBRv2 uses the minimum RTT in recent 5 s to estimate *RTprop*). We also note that the significant throughput degradation starts when the average jitter reaches the path RTT (i.e. 40 ms).

3.7. Summary and implication

We observe that BBRv2 improves inter-protocol fairness and RTT fairness, and reduces retransmission rates under shallow buffers, at the cost of slow responsiveness to bandwidth dynamics and low resilience to random losses.

First, the root cause for the slow responsiveness is that BBRv2 is over-conservative in bandwidth probing. That said, it fails to balance between the aggressiveness in probing for more bandwidth and the fairness against loss-based CCAs. However, recklessly increasing BBRv2's aggressiveness in bandwidth probing may lead BBRv2 to generate overwhelming retransmissions and unfairly share bandwidth with loss-based CCAs. In the next section, we propose BBRv2+, which incorporates delay information to cautiously guide the aggressiveness of bandwidth probing to avoid reducing the fairness against loss-based CCAs. The challenge is how to effectively use this signal and how to avoid being suppressed by other loss-based CCAs in deep-buffered networks as other delay-based CCAs.

Table 1
BBRv2+'s variables capturing delay information and network jitter.

Variable	Functionality
$\text{MinRTT}_{\text{prev_rtt}}$	The minimum RTT measured in the previous RTT round.
$\text{MinRTT}_{\text{curr_rtt}}$	The minimum RTT measured in the current RTT round.
$\text{MinRTT}_{\text{before_probe}}$	Saving the $\text{MinRTT}_{\text{curr_rtt}}$ before entering ProbeUp.
$\text{MinRTT}_{\text{curr_cruise}}$	The minimum RTT measured in the current ProbeCruise state.
$\text{Max}_{4\text{RTT}}(\text{jitter})$	The max_filter tracking the maximum jitter in recent four RTT rounds. Here, the jitter is equivalent to the RTT variation maintained by TCP.

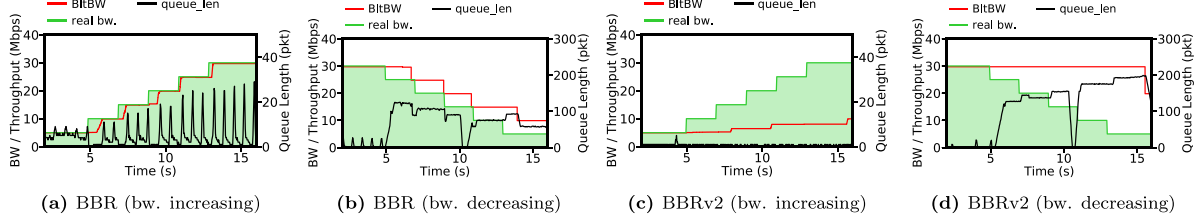


Fig. 12. Responsiveness to bandwidth increases (a, c) and decreases (b, d). The red line represents the $BtBW$ estimation of BBR/BBRv2, and the green line indicates the real bandwidth of the bottleneck link. The dark line shows the dynamics of the bottleneck link's queue length. Note that the spikes of queue length in (a) and (c) are caused by the periodical bandwidth probing of BBR/BBRv2, and the sudden drops of queue_len around 10 s in (b) and (d) are because of probing minimum RTT values. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

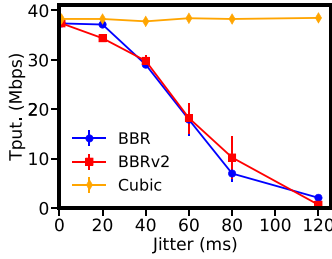


Fig. 13. Average throughput against different levels of jitter. $X = 0$ is equivalent to no jitter.

Second, the resilience to random losses can be improved by raising the loss threshold, α , where the value of α needs to be set according to the bottleneck buffer size.

Last, the throughput degradation of BBR and BBRv2 in high-jitter networks is owned to the underestimation of RT_{prop} , which in turn leads to a smaller estimation of BDP. We propose a compensation mechanism of BDP that makes the estimated BDP close to the real BDP.

4. Design and implementation of BBRv2+

Motivated by our measurement results, we design and implement BBRv2+, in order to address the pitfalls of BBRv2 while maintaining its advantages over BBR (i.e. improved fairness and reduced retransmissions in shallow-buffered networks). The basic idea is to incorporate delay information in BBRv2+'s path model to balance between the aggressiveness in probing for more bandwidth and the fairness against loss-based CCAs. BBRv2+ also compensates the estimated BDP in case of high jitter for improved performance.

4.1. Overview

The architecture of BBRv2+ is shown in Fig. 14. BBRv2+ incorporates delay information in its path model. Specifically, the delay information consists of three state variables (the first three variables listed in Table 1) of minimum RTTs, which reflect the change of queuing delay over time. The delay information facilitates quick responsiveness to bandwidth dynamics. Particularly, a new sub-state, ProbeTry, is added into the ProbeBW state. In ProbeTry, BBRv2+ slightly speeds up to examine if this acceleration will lead to increased

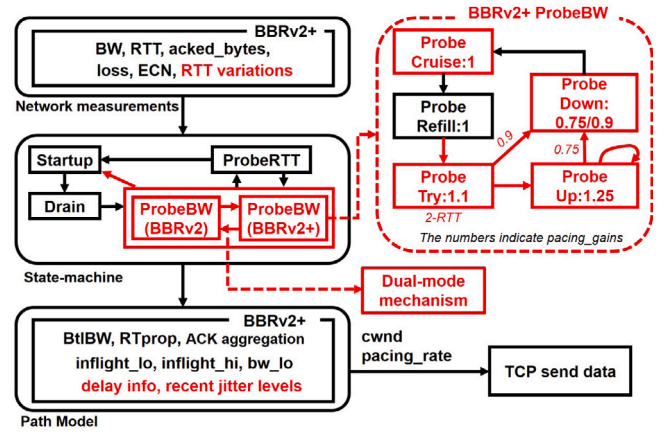


Fig. 14. BBRv2+ architecture. The parts that differs from BBRv2 are highlighted in red color.

RTTs. In the case of increased RTTs, BBRv2+ quits this probing and moves to the ProbeDown state to drain the queue at the bottleneck link; otherwise, it moves to the ProbeUp state to further explore available bandwidth. BBRv2+ also uses the delay information to quickly adapt to bandwidth decreases—it quickly updates its bottleneck bandwidth estimation to the current bandwidth measurement if an obvious increase of RTT is observed when BBRv2+ is not probing for bandwidth.

Like other CCAs that use delay-based signals, BBRv2+ will be suppressed when co-existing with loss-based CCAs under deep buffers [41], as the loss-based CCAs constantly fill the buffer at a bottleneck link, leading BBRv2+ to falsely yield up obtained bandwidth. BBRv2+ uses a dual-mode mechanism that forces BBRv2+ to use BBRv2's state-machine when loss-based CCAs co-exist.

Finally, BBRv2+ uses a BDP compensation mechanism to address the $cwnd$ exhaustion problem caused by network jitter. Our key observation is that in high-jitter networks, BDP will be underestimated because of the underestimation of RT_{prop} . The mechanism compensates BDP by taking the recent RTT variations into consideration; this compensation mitigates the underestimation issue significantly.

4.2. Redesign of the ProbeBW state

In the case of bandwidth increments, BBRv2+ needs to start probing for more bandwidth quickly instead of spending time cruising at the

speed of the current estimated *BtlBW*. Thus, the probing interval needs to be reasonably shortened, which is set to approximately match the probing interval of BBR (8 rounds of RTT). However, if BBRv2+ already sends data at the speed close to bottleneck bandwidth, the probing interval above may result in increased packet losses.

BBRv2+ introduces a two-step probing mechanism by incorporating the delay information as shown in Fig. 14. A new sub-state ProbeTry, which lasts for two RTTs, is inserted before entering ProbeUp in the state machine. In the first RTT of ProbeTry, BBRv2+ slightly increases its *pacing_rate* by increasing *pacing_gain* to 1.1. In the second RTT, BBRv2+ reduces *pacing_gain* to 1.0 and monitors if $\text{MinRTT}_{\text{curr_rtt}}$ is larger than $\gamma \times \text{MinRTT}_{\text{prev_rtt}}$, where MinRTT is measured on the ACKs for the packets sent in the previous round (see Table 1). The rationale of using $\gamma > 1$ is to introduce a relaxing factor to tolerate noises in RTT measurements, where a small γ may lead BBRv2+ to miss some chances to explore bandwidth while a large γ may make BBRv2+ over-aggressive. It is noteworthy that γ is a design parameter and can be tuned by designers.⁶ In our current implementation, we set $\gamma = 1.02$, as such a slight increment (2%) of RTT likely comes from the noises of RTT measurements. If $\text{MinRTT}_{\text{curr_rtt}} > \gamma \times \text{MinRTT}_{\text{prev_rtt}}$, BBRv2+ transits to ProbeDown with *pacing_gain* as 0.9 to drain the queue accumulated during the first RTT of ProbeTry. Otherwise, it enters ProbeUp to probe for more bandwidth.

To further boost the speed of bandwidth discovery, BBRv2+ also incorporates a continuous probing mechanism based on the delay information. Specifically, at the end of ProbeUp, if the $\text{MinRTT}_{\text{curr_rtt}} \leq \gamma \times \text{MinRTT}_{\text{before_probe}}$, BBRv2+ re-enters ProbeUp. The rationale behind this is that there is possibly more free bandwidth capacity as no significant increment of queuing delay arises in the current ProbeUp sub-state.

Algorithm 1: Advance *BtlBW* max_filter

Input : conn: BBRv2+ TCP connection

```

1 target_rtt  $\leftarrow \theta * \text{conn.RTprop}$ 
2 should_advance  $\leftarrow (\text{conn.MinRTT}_{\text{curr\_rtt}} > \text{target\_rtt})$ 
3 if should_advance then
4   | expire_the_oldest_value(conn.BtlBW)
```

We also leverage the delay information to detect bandwidth decrements. If BBRv2+ is in ProbeCruise or ProbeDown, on the receipt of a new ACK in an RTT round, Algorithm 1 is called.⁷ The reason that the algorithm is only applicable in ProbeCruise or ProbeDown is to eliminate the impact on delay variations caused by ProbeTry and ProbeUp sub-states. In Algorithm 1 BBRv2+ expires its current *BtlBW* estimation if $\text{MinRTT}_{\text{curr_rtt}}$ is larger than the recently measured minimum RTT by θ times. θ is a parameter to balance the speed to converge to new bandwidth and the resistance to noises in bandwidth measurements. A small θ may lead to throughput oscillation, while a large θ may reduce BBRv2+'s responsiveness to bandwidth dynamics. We recommend $\theta \in [1.05, 1.15]$ according to our experiences.

4.3. Dual-mode mechanism

Due to the use of delay information to guide the aggressiveness in bandwidth probing, BBRv2+ will be starved by loss-based CCAs under deep buffers, suffering from the similar problem of delay-based CCAs [41]. The root cause is that loss-based CCAs constantly fill the bottleneck buffer, where BBRv2+ falsely treats the increments of RTT as the signals of bandwidth decrements.

⁶ All the design parameters of BBRv2+ in our current implementation are exposed to user-space through the `/sys/module` interfaces, enabling designers to change the parameters without recompiling the kernel module.

⁷ It is called once at maximum for every RTT round to avoid expiring the *BtlBW* estimation too frequently.

Algorithm 2: The dual-mode mechanism

Input : conn: BBRv2+ TCP connection

```

1 if conn.probe_bw_mode = BBRv2+ then
2   | switch_thld  $\leftarrow \lambda_1 * \text{conn.RTprop}$ 
3   | if conn.MinRTTcurr_cruise > switch_thld then
4     | conn.buffer_filling++
5   | else
6     | conn.buffer_filling  $\leftarrow 0$ 
7   | if conn.buffer_filling  $\geq \eta_1$  then
8     | conn.probe_bw_mode  $\leftarrow$  BBRv2
9     | restart_from_startup(conn)
10 else
11   | switch_thld  $\leftarrow \lambda_2 * \text{conn.RTprop}$ 
12   | if conn.MinRTTcurr_cruise  $\leq$  switch_thld then
13     | conn.buffer_empty++
14   | else
15     | conn.buffer_empty  $\leftarrow 0$ 
16   | if conn.buffer_empty  $\geq \eta_2$  then
17     | conn.probe_bw_mode  $\leftarrow$  BBRv2+
```

As BBRv2+ periodically drains the bottleneck buffer, $\text{MinRTT}_{\text{curr_cruise}}$ (see Table 1), the measured minimum RTT during ProbeCruise, should be close to *RTprop* if no loss-based competitor exists. By comparing $\text{MinRTT}_{\text{curr_cruise}}$ with the recorded *RTprop* value, BBRv2+ estimates the existence of loss-based competitors. If loss-based competitors co-exist, BBRv2+ switches to use BBRv2's ProbeBW state, which enables BBRv2+ to co-exist with loss-based CCAs in the same way as BBRv2. Further, if the loss-based competitors no longer exist, BBRv2+ returns back to use the redesigned ProbeBW state. Note that the BBRv2+ does not switch to BBRv2's ProbeBW state if the bottleneck buffer is very shallow, because loss-based CCAs cannot bloat the bottleneck buffer and BBRv2+ will not be starved.

The dual-mode mechanism is detailed in Algorithm 2, which runs at the end of ProbeCruise. If the sender is running in BBRv2+'s ProbeBW state and has not seen RTT samples close to *RTprop* for a number of (η_1) successive ProbeCruise sub-states, it switches to use BBRv2's ProbeBW state and restarts itself from Startup (line 1–9). We note that `restart_from_startup(conn)` in line 9 is a heuristic to quickly regain the bandwidth that has been potentially yielded up to loss-based competitors by BBRv2+ recently. If the sender is using BBRv2's ProbeBW state and it has seen low RTTs for η_2 successive ProbeCruise sub-states, it returns back to use BBRv2+'s ProbeBW state because the competitors are most likely gone. We note that the four parameters in Algorithm 2, λ_1 , λ_2 , η_1 , and η_2 , are to control the sensitivity of BBRv2+ to the co-existence of loss-based CCAs. In practice, we used 1.1, 1.05, 2, and 4 for λ_1 , λ_2 , η_1 , and η_2 respectively. Nevertheless, these parameters can be easily tuned in user space in our current implementation.

4.4. Compensation for BDP estimation

To boost BBRv2+'s performance under high network jitter, BBRv2+ takes network jitter into account when estimating the BDP of a network path. BBRv2+ compensates the BDP estimation with a component proportional to RTT variations when network jitter is high. The compensation mechanism is detailed in Algorithm 3. As instantaneous RTT variations could be very dynamic, to ensure that BBRv2+ can tolerate jitter up to the maximum extent, we use the recently measured maximum RTT variation, $\text{Max}_{4\text{RTT}}(\text{jitter})$ in Table 1, as the estimation of recent jitter. When $\text{Max}_{4\text{RTT}}(\text{jitter})$ exceeds $\mu \times \text{RTprop}$, the estimated *RTprop* is increased to $\text{RTprop} + \text{Max}_{4\text{RTT}}(\text{jitter})$ to mitigate the under-estimation of *RTprop*. We recommend to set μ around 0.5 because the performance of BBRv2 starts to degrade when jitter approaches half of *RTprop* as observed in Section 3.6.

Table 2
Summary of the parameters of BBRv2+.

Parameter	Functionality	Default value
γ	control the aggressiveness of bandwidth probing	1.02
θ	control the sensitivity to the decrease of bandwidth	[1.05, 1.15]
$\lambda_1, \lambda_2, \eta_1, \eta_2$	control the sensitivity to detect if loss-based CCAs co-exist	1.1, 1.05, 2, 4
μ	control the sensitivity to network jitter	0.5

Algorithm 3: Compensating BDP estimation

Input : conn: BBRv2+ TCP connection
Output: the BDP estimation of BBRv2+
1 jitter $\leftarrow$ conn.Max_{4RTT}(jitter)
2 threshold $\leftarrow \mu * \text{conn.RTprop}$
3 fixed_RTprop $\leftarrow \text{conn.RTprop}$
4 **if** jitter > threshold **then**
5 | fixed_RTprop $\leftarrow$ fixed_RTprop + jitter
6 **return** conn.BtlBW * fixed_RTprop

4.5. Implementation

BBRv2+ is implemented as a Linux kernel module (~2100 LoCs), based on Google's BBRv2 alpha kernel module [33]. The parameters of BBRv2+ (summarized in Table 2) are exposed to user-space through the /sys/module interfaces, which allows users to change the parameters according to their need without recompiling the kernel module. The code of BBRv2+ is open-sourced on Github [30] to the research community for further tests and improvements.

5. Evaluation of BBRv2+

In this section, we evaluate BBRv2+ based on both Mininet-based emulation and real-world trace driven emulation. First, we describe our experiment setup in Section 5.1. We then evaluate the benefits of BBRv2+ from the perspectives of the responsiveness to bandwidth changes (Section 5.2) and the resilience to network jitter (Section 5.3). Next, we demonstrate that BBRv2+ is able to keep the advantages of BBRv2 in inter-protocol fairness (Section 5.4), RTT fairness (Section 5.5), and low retransmission rates (Section 5.6). Finally, we evaluate the performance of BBRv2+ through real-world trace driven emulation in Section 5.7.

5.1. Evaluation setup

Two testbeds are used for the evaluation of BBRv2+. One is the Mininet-based testbed used in Section 3, as a controlled environment to evaluate BBRv2+ from various perspectives. The other is based on Mahimahi [29], a trace-driven network emulator that can emulate networks with dynamic bandwidth by replaying bandwidth traces either collected from real-world networks (e.g. Fig. 23) or synthesized manually (e.g. Fig. 17). Other settings of Mahimahi, including delay, packet loss and buffer emulation, are similar to those in Mininet. As shown in Fig. 15, in the Mahimahi-based testbed, a client connects to a server through the emulated network. The physical server running the two testbeds is the same one as that used to evaluate BBRv2 in Section 3. The toolset for data analysis and traffic generation is also the same as that in Section 3.

In all experiments, α (the loss threshold) in BBRv2+ is set to 20% as the value is suitable for most of the buffer sizes according to the results in Section 3.4. Unless specified otherwise, each set of experiments is repeated five times and the averaged results are reported.

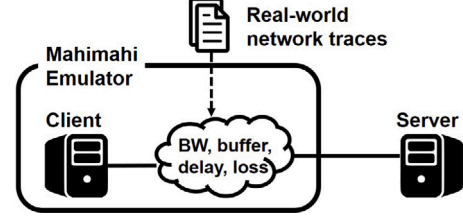


Fig. 15. Mahimahi testbed: the client connects to the server through the network emulated by Mahimahi.

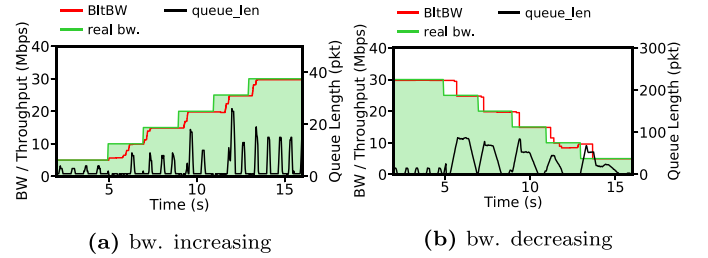


Fig. 16. BBRv2+'s responsiveness to bandwidth increases (a) and decreases (b). The red line represents the BtlBW estimation of BBRv2+, and the green line indicates the real bandwidth of the bottleneck link. The dark line shows the dynamics of the bottleneck link's queue length. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

5.2. Responsiveness to bandwidth dynamics

We use the same settings as that in Section 3.5 to have a microscopic view on how BBRv2+ reacts to bandwidth dynamics. Fig. 16 shows the results.

In Fig. 16(a), when there is no bandwidth increment, BBRv2+ only enters ProbeTry for a very short duration and finishes bandwidth probing very soon, which leads to an instantaneous short standing queue. However, when the bandwidth is increased, BBRv2+ can timely adapt its BtlBW estimation to the real bandwidth. Compared with the results of BBR (Fig. 12(a)) and BBRv2 (Fig. 12(c)), BBRv2+ is capable of utilizing newly available bandwidth as quick as BBR, while the delay information guided probing strategy reduces queuing delay.

In Fig. 16(b), when bandwidth decreases, BBRv2+ notices that the queuing delay is obviously rising up via increased RTTs (see Section 4.2). It expires the old BtlBW estimation and adapts its BtlBW estimation to the available bandwidth. Compared with the results of BBR and BBRv2 in Fig. 12, BBRv2+ adapts its sending rate to the decreased bandwidth much faster, which leads to lower queuing delay.

Next, we compare the responsiveness of Cubic, BBR, BBRv2, and BBRv2+ to bandwidth dynamics in the Mahimahi testbed, using five synthesized traces where the bandwidth changes as step functions, as illustrated in Fig. 17. Following the settings in [6], we set the buffer size to 1.5 MB, the delay to 20 ms, and the loss rate to 0%. For each CCA, we conducted experiments in the five emulated networks. In each experiment, a bulk-transfer TCP flow from the server to the client runs for 30 s and the flow throughput, as well as the sojourn time of each packet in the buffer at the bottleneck link (denoted as queuing delay), are recorded.

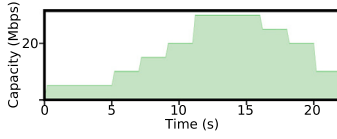


Fig. 17. An example of bandwidth traces with bandwidth changing as a step function.

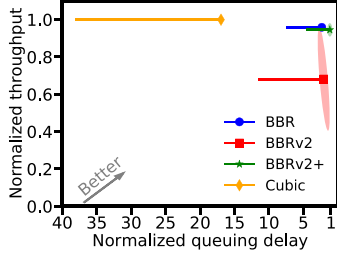


Fig. 18. Normalized throughput and queuing delay of different CCAs (markers: average throughput and queuing delay; left end of each line: 95%tile of queuing delay; ellipses: the standard deviations).

Fig. 18 compares the performance of all considered CCAs. In the figure, for a CCA, we normalize its average queuing delay (over several runs) to the minimum average queuing delay among all considered CCAs in the same emulated network; we also normalize its average throughput (over several runs) to the maximum average throughput among all considered CCAs. In addition, we report the 95%tile queuing delay of each CCA, which is also normalized to the minimum average queuing delay observed in the same network. Then, the averages and deviations of the normalized values across all emulated networks are plotted. We can see from the figure that, compared with BBRv2, BBRv2+ achieves significantly higher throughput and lower queuing delay. On the other hand, compared with BBR, BBRv2+ achieves lower queuing delay at the cost of slightly lower throughput. These observations stem from the facts that: (1) BBRv2+ probes for bandwidth at a frequency similar to BBR's one, thus, achieving high bandwidth utilization as BBR does; (2) BBRv2+ adapts its sending rate to decreased bandwidth faster than BBR as it quickly updates its *BtlBW* estimation upon increased queuing delay.

5.3. Resilience to network jitter

Next, we evaluate how network jitter affects BBRv2+'s performance, using the same settings as in Section 3.6. The throughput of BBRv2+, Cubic, BBR and BBRv2 under various levels of jitter are shown in Fig. 19(a). Different from BBR and BBRv2, the throughput of BBRv2+ does not degrade when the network jitter becomes larger. Fig. 19(b) further plots the average inflight bytes of four CCAs; the results confirm that the BDP compensation mechanism succeeds to compensate *cwnd* in high-jitter networks. Nevertheless, the throughput of BBRv2+ is slightly lower than Cubic. The reason is that our compensation to BDP is conservative; in contrast, Cubic's *cwnd* can grow far beyond the real BDP as it is not affected by network jitter.

5.4. Inter-protocol fairness

The inter-protocol fairness of BBRv2+ is evaluated using the same settings as in Section 3.2.1. We test BBRv2+ with and without the dual-mode mechanism (see Section 4.3) to study the impact of this mechanism on inter-protocol fairness. The results are shown in Fig. 20. Several observations are notable.

Firstly, the results demonstrate the efficacy of the dual-mode mechanism. In Fig. 20(a), we can observe that BBRv2+ without the dual-mode mechanism is starved by Cubic in deep-buffered cases. This is because

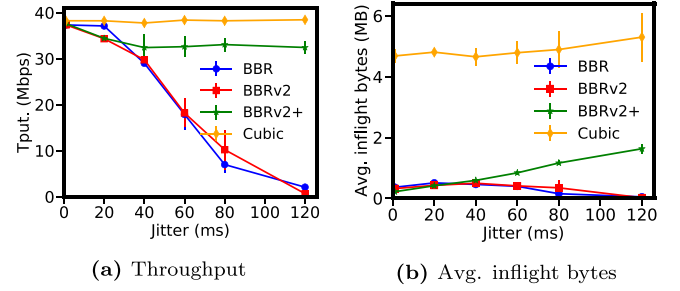


Fig. 19. Resilience to network jitter (markers: averages, bars: standard deviations).

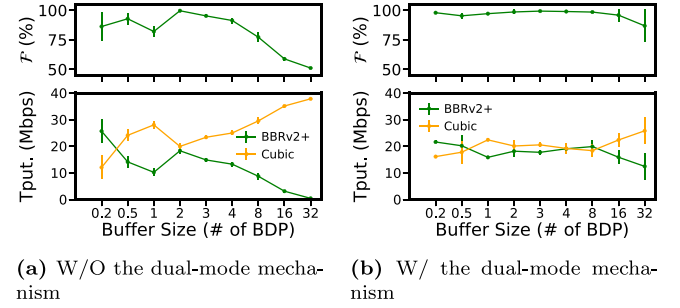


Fig. 20. BBRv2+: Inter-protocol fairness under different buffer sizes (markers: averages, bars: standard deviations).

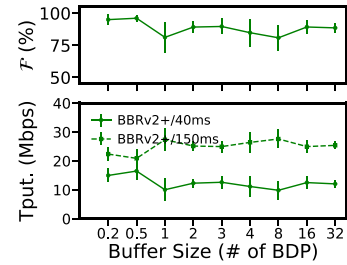


Fig. 21. BBRv2+: RTT fairness under different buffer sizes (markers: averages, bars: standard deviations).

BBRv2+ falsely treats the RTT increments caused by Cubic as the signals of bandwidth shrinking. The problem is eliminated by the dual-mode mechanism as shown in Fig. 20(b).

Second of all, compared with the results of BBRv2(20%, 0.3) in Fig. 11, we can observe that: (1) BBRv2+ provides better inter-protocol fairness than BBRv2(20%, 0.3) under an extremely shallow buffer (i.e. $0.2 \times \text{BDP}$); (2) BBRv2+'s inter-protocol fairness is similar to that of BBRv2(20%, 0.3) under other buffer sizes. The reason for the better inter-protocol fairness of BBRv2+ under an extremely shallow buffer is that BBRv2+ does not enter ProbeUp, which is more aggressive than ProbeTry, due to the two-step probing mechanism (see Section 4.2) while BBRv2(20%, 0.3) periodically enters ProbeUp.

Lastly, compared with the results of BBR (Fig. 5(a)) and BBRv2 (Fig. 5(b)), BBRv2+ performs no worse than the better one among BBR and BBRv2 under different buffer sizes. The reasons are three-fold: (1) under shallow buffers, BBRv2+ achieves similar inter-protocol fairness to that of BBRv2 thanks to its cautiously aggressive bandwidth probing strategy (see Section 4.2); (2) under moderate buffers, BBRv2+ is close to BBRv2(20%, 0.3) that has better inter-protocol fairness than BBRv2 in these cases as explained in Section 3.4; (3) under deep buffers, the three CCAs perform closely as they all upper bound *cwnd* around $2 \times \text{BDP}$, thus, unable to beat loss-based CCAs.

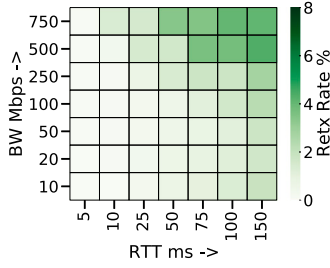


Fig. 22. BBRv2+: retransmission rates (100 KB buffer).

5.5. RTT fairness

Next, we evaluate the RTT fairness of BBRv2+ using the same setting as in Section 3.2.2. The results are presented in Fig. 21. Compared with the results of BBR (Fig. 6(a)) and BBRv2 (Fig. 6(b)), BBRv2+ has better RTT fairness than BBR and behaves close to BBRv2. The results are expected as the mechanisms in BBRv2 that improves the RTT fairness over BBR (see Section 3.2.2) remain unchanged in BBRv2+.

5.6. Retransmissions in shallow-buffered networks

Using the same setting as in Section 3.3, we evaluate the retransmissions in shallow-buffered networks in Fig. 22. We observe that when the buffer is extremely shallow (e.g. $0.02 \times \text{BDP}$ when the bandwidth is 500 Mbps and the RTT is 75 ms), BBRv2+ incurs more retransmissions than BBRv2. This is because the bandwidth probing frequency of BBRv2+ is higher than that of BBRv2. Although BBRv2+ uses a relatively small *pacing gain* (1.1), when it starts to probe for more bandwidth in ProbeTry, it still causes buffer overflow if the network buffer is extremely shallow. However, compared with the results of BBR in Fig. 7(a), BBRv2+ reduces retransmissions significantly.

5.7. Real-world trace driven emulation

Finally, we evaluate BBRv2+ in the Mahimahi testbed, where the input bandwidth traces are collected in real-world networks. We compare BBRv2+ with Cubic, BBR, BBRv2, and Orca [6]. Orca⁸ is used for comparison as a representative of the state-of-the-art learning-based CCAs.

Trace collection: We collected bandwidth traces from WiFi and LTE networks, using *saturatr* [42]. In total, 20 bandwidth traces are collected, half of which were collected when the collector was stationary to the base station (LTE) or the Access Point (WiFi), and the other half were collected when the collector was moving at high speeds (i.e. in vehicles or on high-speed rails). In stationary scenarios, the network bandwidth was usually stable, while in high-mobility scenarios, the bandwidth fluctuated greatly. Examples of stationary and high-mobility traces are shown in Fig. 23. For each bandwidth trace, we also measured the packet loss rate and delay of its corresponding network via ping.

Experimental setup: Using the collected bandwidth traces along with the information of packet loss rates and delays, we emulate various network environments, where their buffer sizes are fixed at 1.5 MB as the setup in Section 5.2. In each run of the experiments, a bulk-transfer TCP flow from the server to the client runs for 30 s.

⁸ We directly used the model trained by the authors in our experiments.

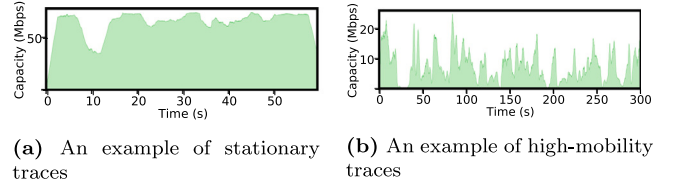


Fig. 23. Examples of bandwidth traces used in our trace-driven evaluation.

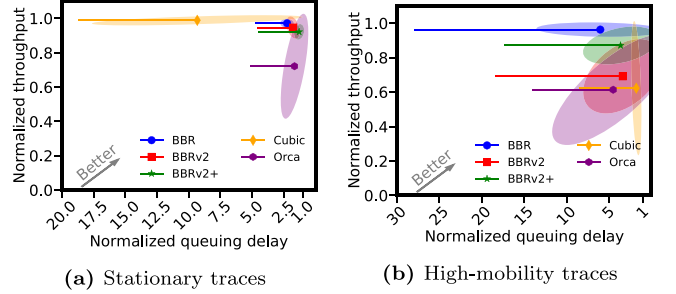


Fig. 24. Normalized throughput and queuing delay of different CCAs (markers: average throughput and queuing delay, left end of each line: 95%ile of queuing delay, ellipses: the standard deviations of the throughput and queuing delay).

Results: Using the same metrics as in Section 5.2 (normalized throughput and queuing delay), the results for the stationary and high-mobility scenarios are shown in Fig. 24(a) and Fig. 24(b) respectively.

In stationary scenarios, BBR, BBRv2, and BBRv2+ perform very close to each other because the bandwidth is usually stable. Cubic shows slightly better throughput for most of the time at the cost of high queuing delay. Orca has the lowest throughput probably because the network scenario where the model was trained is different from the emulated network environments in our experiments, which also demonstrates the limitation of learning-based CCAs.

In high-mobility scenarios, BBR and BBRv2+ achieve the highest and the second-highest throughput respectively. Meanwhile, BBRv2 and Cubic fail to achieve consistently high throughput across different high-mobility network environments. Compared with BBRv2+, BBR achieves higher throughput at the cost of higher queuing delay as it is more aggressive. Orca fails to achieve consistently high throughput and low queuing delay in high-mobility scenarios; the results of Orca raise a concern on the generalization ability of learning-based CCAs.

5.8. Summary of experimental results

We can conclude from the above experiments that BBRv2+ succeeds to balance the aggressiveness of bandwidth probing and the fairness against loss-based CCAs. With such a balance, BBRv2+ achieves higher throughput and lower delay than BBRv2 in scenarios where the bandwidth fluctuates, while keeping the advantages of BBRv2 with regard to inter-protocol fairness and reduced retransmissions under shallow buffers. Moreover, the dual-mode mechanism enables BBRv2+ to co-exist with loss-based CCAs under deep buffers and the compensation mechanism for BDP estimation efficiently enhances the performance of BBRv2+ under high network jitter.

6. Related work

BBR evaluation: Since BBR [12] was released by Google in 2016, it has been examined under various network conditions by researchers [3,4,15–18]. It has been found that BBR is unfair when sharing a bottleneck link with Cubic flows or BBR flows with different RTTs [3,4,15,18]. Besides, in shallow-buffered networks, BBR can lead to massive

retransmissions [4]. Moreover, the throughput of BBR collapses when network jitter is high, either in experimental emulation [17] or networks in high-speed train scenarios [27].

BBR enhancement: The issues of BBR identified by empirical studies motivated optimizations on BBR from various perspectives. For instance, [17,27] proposed several modifications of *RTprop* estimation to counter against network jitter; [18,43,44] improved BBR's RTT fairness; [45,46] improved BBR's inter-protocol fairness with loss-based CCAs; [47] reduced BBR's aggressiveness in shallow-buffered networks to suppress the unnecessary retransmissions.

BBRv2 evaluation: Google proposed BBRv2 [19] to solve the problems identified in BBR. In Google's early tests [19,21], BBRv2 showed better inter-protocol fairness with loss-based CCAs and reduced retransmissions in shallow-buffered networks. There are also several evaluations on BBRv2 [2,20,22,23]. Gomez et al. [20] and Nadagiri et al. [22] studied the inter-protocol fairness and RTT fairness of BBRv2. Kfoury et al. [23] found that BBRv2 eliminates the massive retransmissions in shallow-buffered networks. Song et al. [2] found that BBRv2 cannot quickly utilize the newly available link capacity when the bottleneck bandwidth increases.

Our work differs from the above studies in two perspectives: (1) from the measurement perspective, we not only systematically evaluate the performance of BBRv2 under various network conditions, but also analyze the reasons behind the observed performance issues; (2) from the optimization perspective, we propose BBRv2+ that addresses the shortcomings of BBRv2 while barely sacrificing its advantages in fairness and reduced retransmissions.

7. Conclusion

In this paper, we first comprehensively evaluate BBRv2, reveal its benefits and shortcomings over BBR, and analyze the reasons behind the observations. We then propose BBRv2+ that incorporates delay information in its path model and redesigns the ProbeBW state to balance between the aggressiveness of bandwidth probing and the fairness against loss-based CCAs. In doing so, BBRv2+ takes the advantages from both BBR and BBRv2, i.e. achieving both high performance and good TCP fairness. Extensive experiments demonstrate that BBRv2+ significantly improves the performance over BBRv2, especially in high-mobility scenarios, while barely sacrificing the advantages of BBRv2. In addition, with the BDP compensation mechanism, BBRv2+ barely loses throughput under high network jitter.

CRedit authorship contribution statement

Furong Yang: Writing – original draft, Software, Methodology, Visualization, Investigation. **Qinghua Wu:** Conceptualization, Validation, Writing – review & editing. **Zhenyu Li:** Supervision, Validation, Funding acquisition, Writing – review & editing. **Yanmei Liu:** Resources, Writing – review & editing. **Giovanni Pau:** Writing – review & editing. **Gaogang Xie:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was partially supported by Beijing Natural Science Foundation (JQ20024), Natural Science Foundation of China (U20A20180, 62072437), and CAS-Austria Joint Project (171111KYSB20200001). Giovanni Pau's work was partially supported by Emilia-Romagna region (Italy), POR-FESR 2018–2020 Project LiBER and the Macao Polytechnic Institute project (RP/ESCA-01/2020).

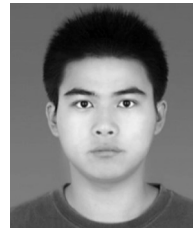
References

- [1] S.H. Yeganeh, P. Jha, Y. Seung, L. Hsiao, M. Mathis, V. Jacobson, BBR updates: Internal deployment, code, draft plans, 2021, p. 8.
- [2] Y.-J. Song, G.-H. Kim, I. Mahmud, W.-K. Seo, Y.-Z. Cho, Understanding of BBRv2: Evaluation and comparison with BBRv1 congestion control algorithm, *IEEE Access* 9 (2021) 37131–37145.
- [3] D. Scholz, B. Jaeger, L. Schwaighofer, D. Raumer, F. Geyer, G. Carle, Towards a deeper understanding of TCP BBR congestion control, in: 2018 IFIP Networking Conference (IFIP Networking) And Workshops, 2018, pp. 1–9.
- [4] M. Hock, R. Bless, M. Zitterbart, Experimental evaluation of BBR congestion control, in: 2017 IEEE 25th International Conference On Network Protocols, ICNP, 2017, pp. 1–10.
- [5] V. Jacobson, Congestion avoidance and control, *ACM SIGCOMM Comput. Commun. Rev.* 18 (4) (1988) 314–329.
- [6] S. Abbasi, C.-Y. Yen, H.J. Chao, Classic meets modern: A pragmatic learning-based congestion control for the internet, in: Proceedings of SIGCOMM'20, 2020, pp. 632–647.
- [7] K. Winstein, H. Balakrishnan, Tcp ex machina: Computer-generated congestion control, *ACM SIGCOMM Comput. Commun. Rev.* 43 (4) (2013) 123–134.
- [8] N. Jay, N. Rotman, B. Godfrey, M. Schapira, A. Tamar, A deep reinforcement learning perspective on internet congestion control, in: ICML, 2019, pp. 3050–3059.
- [9] M. Dong, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, B. Godfrey, M. Schapira, PCC vivace: Online-learning congestion control, in: 15th USENIX Symposium On Networked Systems Design And Implementation, NSDI 18, 2018, pp. 343–356.
- [10] F.Y. Yan, J. Ma, G.D. Hill, D. Raghavan, R.S. Wahby, P. Levis, K. Winstein, Pantheon: the training ground for internet congestion-control research, in: 2018 USENIX Annual Technical Conference, USENIX ATC 18, 2018, pp. 731–743.
- [11] I. Rhee, L. Xu, S. Ha, A. Zimmermann, L. Eggert, R. Scheffenecker, CUBIC for fast long-distance networks, 2018, RFC 8312.
- [12] N. Cardwell, Y. Cheng, C.S. Gunn, S.H. Yeganeh, V. Jacobson, BBR: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time, *Queue* 14 (5) (2016) 20–53.
- [13] A. Mishra, X. Sun, A. Jain, S. Pande, R. Joshi, B. Leong, The great internet tcp congestion control census, *Proc. ACM Measur. Anal. Comput. Syst.* 3 (3) (2019) 1–24.
- [14] L. Kleinrock, Power and deterministic rules of thumb for probabilistic problems in computer communications, in: Proceedings Of The International Conference On Communications, vol. 3, 1979, pp. 1–43.
- [15] Y. Cao, A. Jain, K. Sharma, A. Balasubramanian, A. Gandhi, When to use and when not to use BBR: An empirical analysis and evaluation study, in: Proceedings Of The Internet Measurement Conference, ACM, Amsterdam Netherlands, 2019, pp. 130–136.
- [16] R. Ware, M.K. Mukerjee, S. Seshan, J. Sherry, Modeling BBR's interactions with loss-based congestion control, in: Proceedings Of The Internet Measurement Conference, ACM, Amsterdam Netherlands, 2019, pp. 137–143.
- [17] R. Kumar, A. Koutsafis, F. Fund, G. Naik, P. Liu, Y. Liu, S. Panwar, TCP BBR for ultra-low latency networking: Challenges, analysis, and solutions, in: 2019 IFIP Networking Conference, IFIP Networking, 2019, pp. 1–9.
- [18] S. Ma, J. Jiang, W. Wang, B. Li, Fairness of congestion-based congestion control: Experimental evaluation and analysis, 2017.
- [19] N. Cardwell, Y. Cheng, S.H. Yeganeh, I. Swett, V. Vasiliev, P. Jha, Y. Seung, M. Mathis, V. Jacobson, BBR v2: A model-based congestion control, 2019.
- [20] J. Gomez, E. Kfoury, J. Crichigno, E. Bou-Harb, G. Srivastava, A performance evaluation of TCP BBRv2 alpha, in: 2020 43rd International Conference On Telecommunications And Signal Processing (TSP), IEEE, Milan, Italy, 2020, pp. 309–312.
- [21] N. Cardwell, Y. Cheng, S.H. Yeganeh, P. Jha, Y. Seung, I. Swett, V. Vasiliev, B. Wu, M. Mathis, V. Jacobson, BBR v2: A model-based congestion control IETF 105 update, 2019.
- [22] A. Nadagiri, M.P. Tahiliani, V. Misra, K.K. Ramakrishnan, BBRv1 vs BBRv2: Examining performance differences through experimental evaluation, in: 2020 IEEE International Symposium On Local And Metropolitan Area Networks, LANMAN, IEEE, Orlando, FL, USA, 2020, pp. 1–6.
- [23] E.F. Kfoury, J. Gomez, J. Crichigno, E. Bou-Harb, An emulation-based evaluation of TCP BBRv2 Alpha for wired broadband, *Comput. Commun.* 161 (2020) 212–224.
- [24] M. Zhang, M. Polese, M. Mezzavilla, J. Zhu, S. Rangan, S. Panwar, M. Zorzi, Will TCP work in mmWave 5G cellular networks? 2018.
- [25] D. Chitmall, K. Kondep, L. Valcarengi, M. Tornatore, B. Mukherjee, 5G fronthaul-latency and jitter studies of CPRI over ethernet, *IEEE/OSA J. Opt. Commun. Netw.* 9 (2) (2017) 172–182.
- [26] J.D. Beshay, A.T. Nasrabadi, R. Prakash, A. Francini, Link-coupled TCP for 5G networks, in: 2017 IEEE/ACM 25th International Symposium On Quality Of Service, IWQoS, 2017, pp. 1–6.

- [27] J. Wang, Y. Zheng, Y. Ni, C. Xu, F. Qian, W. Li, W. Jiang, Y. Cheng, Z. Cheng, Y. Li, X. Xie, Y. Sun, Z. Wang, An active-passive measurement study of TCP performance over LTE on high-speed rails, in: The 25th Annual International Conference On Mobile Computing And Networking, ACM, Los Cabos Mexico, 2019, pp. 1–16.
- [28] Mininet: An instant virtual network on your laptop (or other PC) - Mininet, <http://mininet.org/>.
- [29] R. Netravali, A. Sivaraman, S. Das, A. Goyal, K. Winstein, J. Mickens, H. Balakrishnan, Mahimahi: Accurate record-and-replay for HTTP, in: ATC, 2015, p. 14.
- [30] BBRv2+, <https://github.com/yangfurong/BBRv2plus>.
- [31] M. Alizadeh, A. Greenberg, D.A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, M. Sridharan, Data center tcp (dctcp), in: Proceedings Of The ACM SIGCOMM 2010 Conference, 2010, pp. 63–74.
- [32] N. Cardwell, Y. Cheng, C.S. Gunn, S.H. Yeganeh, I. Swett, J. Iyengar, V. Vasiliev, P. Jha, Y. Seung, K. Yang, M. Mathis, V. Jacobson, BBR congestion control work at google IETF 102 update, 2018.
- [33] google/bbr, <https://github.com/google/bbr/tree/v2alpha>.
- [34] netem, <https://www.linux.org/docs/man8/tc-netem.html>.
- [35] esnet/iperf, <https://github.com/esnet/iperf>.
- [36] tcpdump, <https://www.tcpdump.org/>.
- [37] tcptrace, <https://linux.die.net/man/1/tcptrace>.
- [38] tc, <https://man7.org/linux/man-pages/man8/tc.8.html>.
- [39] S. Floyd, Metrics for the evaluation of congestion control mechanisms, 2008, RFC 5166.
- [40] B. Huffaker, M. Fomenkov, D.J. Plummer, D. Moore, K. Claffy, et al., Distance metrics in the internet, in: Proc. Of IEEE International Telecommunications Symposium, ITS, 2002.
- [41] R. Al-Saadi, G. Armitage, J. But, P. Branch, A survey of delay-based and hybrid TCP congestion control algorithms, IEEE Commun. Surv. Tutor. 21 (4) (2019) 3609–3638.
- [42] K. Winstein, A. Sivaraman, H. Balakrishnan, Stochastic forecasts achieve high throughput and low delay over cellular networks, in: 10th USENIX Symposium On Networked Systems Design And Implementation, NSDI 13, 2013, pp. 459–471.
- [43] M. Yang, P. Yang, C. Wen, Q. Liu, J. Luo, L. Yu, Adaptive-BBR: Fine-grained congestion control with improved fairness and low latency, in: 2019 IEEE Wireless Communications And Networking Conference, WCNC, 2019, pp. 1–6.
- [44] G.-H. Kim, Y.-Z. Cho, Delay-aware BBR congestion control algorithm for RTT fairness improvement, IEEE Access 8 (2020) 4099–4109.
- [45] Y. Zhang, L. Cui, F.P. Tso, Modest BBR: Enabling better fairness for BBR congestion control, in: 2018 IEEE Symposium On Computers And Communications, ISCC, 2018, pp. 646–651.
- [46] Y.-J. Song, G.-H. Kim, Y.-Z. Cho, BBR-CWS: Improving the inter-protocol fairness of BBR, Electronics 9 (5) (2020) 862.
- [47] I. Mahmud, G.-H. Kim, T. Lubna, Y.-Z. Cho, BBR-ACD: BBR with advanced congestion detection, Electronics 9 (1) (2020) 136.



Furong Yang is currently a Ph.D. candidate at ICT/CAS. He is also in a joint doctoral program between the University of Chinese Academy of Sciences and Sorbonne University since 2018. His main research interests lie in TCP congestion control, multipath transport protocol, and Internet measurement.



Qinghua Wu received the Ph.D. degree from ICT/CAS in 2015. He is currently an Associate Researcher at ICT/CAS. His research interests lie in network transport protocol and Internet measurement.



Zhenyu Li received the Ph.D. degree from ICT/CAS in 2009. He is currently a Full Professor at ICT/CAS and an Adjunct Professor at the University of CAS. His research interests include Internet architecture and Internet measurement.



Yanmei Liu received the B.E. degree in microelectronic technology from the University of Electronic Science and Technology of China, in 2013. She is currently a senior Research and Development (R&D) Expert in Alibaba Group. Her research focuses on QUIC, multipath QUIC, and transport protocol design.



Giovanni Pau is a Full Professor in the Department of Computer Science and Engineering (DISI) at the University of Bologna; He also serves as an Adjunct Professor at UCLA. He holds the Italian Laura in Computer Science and a Ph.D. in Computer Engineering awarded by the University of Bologna in 1998 and 2002 respectively. Before joining the University of Bologna, Dr. Pau served as ATOS/Renault smart mobility chair of excellence Professor at the Sorbonne Université, Paris France. Dr. Pau's core research interests are in Network Systems with a focus on Vehicular Networks and pervasive mobile sensor systems.



Gaogang Xie (Member, IEEE) received the B.S. degree in physics and the M.S. and Ph.D. degrees in computer science from Hunan University in 1996, 1999, and 2002, respectively. He is currently a Professor with the Computer Network Information Center (CNIC), Chinese Academy of Sciences (CAS), and the University of Chinese Academy of Sciences (UCAS), and the Vice President of CNIC. His research interests include Internet architecture, packet processing and forwarding, and Internet measurement.